LEETCODE + CO-OP INTERVIEW PREPARATION NOTES

Student: First-Year Software Engineering Student, uOttawa
Goal: Prepare for software engineering co-op interviews and build strong problem-solving recall.

MAIN STRATEGY

The goal is not just to solve many LeetCode problems. The goal is to understand patterns, remember mistakes, and explain solutions clearly in interviews.

For each problem, I should remember:

1. What pattern the problem uses.
2. How to recognize that pattern.
3. The main trick or key idea.
4. The mistake I made.
5. The optimal solution.
6. The time and space complexity.
7. How to explain it in an interview.

CO-OP STRATEGY

To maximize my chances of getting a software engineering co-op, I should focus on four main areas:

1. Resume

My resume must clearly show my strongest projects, technical skills, GitHub, deployed links, and impact. Since I already have full-stack projects using tools like AWS, Spring Boot, and React, I should make sure each project has bullet points that explain what I built, what problem it solves, and what technologies I used.

2. Projects

For each major project, I should have:

- A clean GitHub repository
- A strong README
- Screenshots
- A deployed demo link if possible
- A clear explanation of the tech stack
- Features listed clearly
- Any challenges I solved

In interviews, I should be ready to explain:

- Why I built the project
- How the frontend works
- How the backend works
- How the database works
- How authentication works, if used
- What I would improve next

3. Networking

I should not only rely on online applications. I should message people on LinkedIn, especially:

- uOttawa alumni
- Software engineering students in upper years
- Previous co-op students
- Developers at companies I like
- Recruiters
- People working in Ottawa, Toronto, government, banks, startups, and tech companies

LinkedIn message template:

Hi [Name],

I'm a first-year Software Engineering student at uOttawa preparing for my first co-op.

I noticed you worked at [Company], and I'd really appreciate hearing about your experience and any advice you have for someone trying to break into software engineering internships.

Thanks,
[Your Name]

The goal is not to immediately ask for a job. The goal is to build a connection, learn from them, and possibly get advice or a referral later.

4. Applications

I should apply broadly, not only to big tech companies.

Good places to apply:

- uOttawa co-op portal
- LinkedIn
- Indeed
- company websites
- government jobs
- banks
- startups
- consulting companies
- local Ottawa companies
- Toronto tech companies
- remote internships
- research assistant roles
- QA/testing roles
- software developer intern roles
- web developer intern roles

For a first co-op, experience matters more than prestige.

LEETCODE STRATEGY

For first internship interviews, I should focus on mastering patterns instead of randomly solving problems.

Most important patterns:

1. Hash Map
2. Array
3. String
4. Two Pointers
5. Sliding Window
6. Linked List
7. Stack
8. Queue
9. Binary Search
10. Tree DFS
11. Tree BFS
12. Recursion
13. Heap
14. Graphs
15. Dynamic Programming

Daily goal:

Solve at least 1 problem per day.

Weekly goal:

- 5 to 7 LeetCode problems
- 2 to 5 job applications
- 2 LinkedIn networking messages
- 1 resume improvement
- 1 project improvement
- 1 mock interview or self-recorded explanation

PROBLEM NOTE FORMAT

For every LeetCode problem, I should write notes using this format:

Problem Title — Date

Pattern:
Write the main pattern.

Recognition Clue:
Write how I can recognize this type of problem in the future.

Key Idea:
Write the main trick that makes the solution work.

Mistake I Made:
Write the exact mistake I made.

Optimal Solution:
Explain the correct solution clearly.

Complexity:

Time:
Space:

Interview Explanation:
Write how I would explain the solution out loud in an interview.

Similar Problems:
Write related problems that use the same idea.

QUICK INTERVIEW TIPS

When explaining a solution, I should say:

1. First, I'll explain the brute force idea.
2. Then, I'll explain why it is inefficient.
3. Then, I'll explain the optimized pattern.
4. Then, I'll walk through the algorithm.
5. Then, I'll give time and space complexity.

I should not silently code. I should explain my thinking.

Good interview phrase:

"I'll first think about the simple approach, then optimize it using the pattern I recognize."

Another good phrase:

"The main issue with brute force is that it repeats work, so I'll use a data structure to remember information as I scan."

IMPORTANT MINDSET

First co-op is usually the hardest one to get. I should not get discouraged if I apply a lot and hear nothing back. My job is to keep improving my resume, keep networking, keep applying, and keep getting better at explaining my projects and LeetCode solutions.

I already have a strong foundation because I have projects, GitHub, deployed work, and I'm starting interview preparation early.

The goal is consistency.

==========
TWO SUM
==========

Pattern:
Hash Map, Array

Recognition Clues:
- Find two numbers that add up to a target
- Need indices rather than values
- Fast lookup required
- Brute force would use nested loops

When I see:
"Find pair"
"Find complement"
"Target sum"

Think:
Hash Map

Key Insight:

needed = target - current

Instead of searching for the second number every time, calculate the complement and check if it has already been seen.

Mistake I Made:
Did not use a hash map.
Started thinking toward a brute force $O(n^2)$ solution.

Optimal Solution:
1. Create a hash map.
2. For each number:
   - Calculate needed = target - current
   - Check if needed exists in the map
   - If yes, return answer
   - Otherwise store current number

Complexity:
Time: O(n)
Space: O(n)

Interview Explanation:
"As I scan through the array, I calculate the complement needed to reach the target. If I've already seen that complement, I found my answer. Otherwise I store the current value for future lookups."

Similar Problems:
- Contains Duplicate
- Two Sum II
- 3Sum

- 4Sum

Memory Trick:
Target = Current + Needed
Needed = Target - Current

==========
MERGE TWO SORTED LISTS
==========

Pattern:
Linked List
Two Pointers
Recursion

Recognition Clues:
- Two sorted linked lists
- Merge while preserving order
- Compare current nodes

Think:
Compare heads
Attach smaller
Move pointer

Key Insight:
Only compare the current nodes.
Attach the smaller node.
Move only the pointer that was chosen.

Mistake I Made:
Confused pointer movement.
Forgot to properly reconnect nodes after choosing one.

Optimal Solution:
1. Create a dummy node.
2. Compare list1.val and list2.val.
3. Attach the smaller node.
4. Move that pointer.
5. Continue until one list ends.
6. Attach the remaining nodes.

Complexity:
Time: O(n + m)
Space: O(1) iterative
Space: O(n + m) recursive

Interview Explanation:
"I repeatedly compare the current nodes of both lists, attach the smaller node to the merged list, and move the corresponding pointer."

Similar Problems:
- Merge K Sorted Lists

- Sort List
- Reverse Linked List

Memory Trick:
Compare
Attach
Move

Compare
Attach
Move

==========
CLIMBING STAIRS
==========

Pattern:
Dynamic Programming
Memoization

Recognition Clues:
- Count number of ways
- Number of possibilities
- Repeated subproblems

Think:
Dynamic Programming

Key Insight:

dp[n] = dp[n-1] + dp[n-2]

To reach stair n:
- Come from n-1
- Come from n-2

This is essentially Fibonacci.

Mistake I Made:
Used recursion without memoization.
Repeated calculations many times.

Optimal Solution:
Store previous answers and build upward.

Complexity:
Time: O(n)
Space: O(1)

Interview Explanation:
"Each stair can be reached from either the previous stair or the stair before that, so the total ways are the sum of those two previous results."

Similar Problems:
- Fibonacci
- House Robber
- Min Cost Climbing Stairs

Memory Trick:
Current = Previous + Previous Previous

==========
REMOVE DUPLICATES FROM SORTED LIST
==========

Pattern:
Linked List

Recognition Clues:
- Sorted linked list
- Remove duplicates
- Keep only one copy

Think:
Skip duplicate nodes

Key Insight:
Because the list is sorted, duplicates always appear beside each other.

Mistake I Made:
Forgot to update current.next when duplicates appeared.

Optimal Solution:
If:

current.val == current.next.val

Then:

current.next = current.next.next

Otherwise move forward.

Complexity:
Time: O(n)
Space: O(1)

Interview Explanation:
"Since duplicates are adjacent in a sorted list, I can remove them by reconnecting the current node to the next distinct node."

Similar Problems:
- Remove Linked List Elements
- Remove Duplicates from Sorted Array

Memory Trick:

Same value?

Skip node.

==========
SAME TREE
==========

Pattern:
Tree
DFS
Recursion

Recognition Clues:
- Compare two trees
- Check if trees are identical

Think:
DFS

Key Insight:
Both trees must satisfy:

1. Same value
2. Same left subtree
3. Same right subtree

Mistake I Made:
Only compared node values.
Forgot to compare left and right subtrees.

Optimal Solution:
Recursively compare:
- Value
- Left subtree
- Right subtree

Complexity:
Time: O(n)
Space: O(h)

Interview Explanation:
"Two trees are identical only if the current nodes match and both corresponding subtrees also match."

Similar Problems:
- Symmetric Tree
- Subtree of Another Tree

Memory Trick:
Value same
Left same
Right same

==========
MAXIMUM DEPTH OF BINARY TREE
==========

Pattern:
Tree
DFS
Recursion

Recognition Clues:
- Height
- Depth
- Longest path

Key Insight:

Depth = 1 + max(leftDepth, rightDepth)

Mistake I Made:
Forgot to add +1 for the current node.

Optimal Solution:
1. Recursively calculate left depth.
2. Recursively calculate right depth.
3. Take maximum.
4. Add one.

Complexity:
Time: O(n)
Space: O(h)

Interview Explanation:
"The depth of a node is one plus the larger depth of its two children."

Similar Problems:
- Balanced Binary Tree
- Diameter of Binary Tree

Memory Trick:
Current node counts

+1

==========
MINIMUM DEPTH OF BINARY TREE
==========

Pattern:
Tree
DFS
BFS

Recognition Clues:
- Minimum depth
- Nearest leaf
- Shortest root-to-leaf path

Key Insight:
This is NOT maximum depth.

Answer = shortest path from root to a leaf.

Mistake I Made:
Used maximum-depth logic.

Optimal Solution:
Find the shortest valid root-to-leaf path.
BFS often makes this easiest to visualize.

Complexity:
Time: O(n)
Space: O(h) DFS

Interview Explanation:
"The minimum depth is the shortest path from the root to any leaf node, not simply the smaller subtree depth."

Similar Problems:
- Maximum Depth
- Balanced Binary Tree

Memory Trick:
Nearest leaf wins

==========
PATH SUM
==========

Pattern:
Tree
DFS
Recursion

Recognition Clues:
- Root-to-leaf path
- Target sum
- Tree traversal
- Need to determine if a path exists

Think:
DFS + Running Sum

Key Insight:

At each node:

remainingSum = targetSum - currentNode.val

When reaching a leaf:

remainingSum == 0

means the path exists.

Mistake I Made:
Forgot to subtract the current node value while traversing.

Optimal Solution:
1. Visit node.
2. Subtract node value from remaining target.
3. If leaf node:
   - Check whether remaining target equals zero.
4. Recursively explore left and right children.

Complexity:
Time: O(n)
Space: O(h)

Interview Explanation:
"As I traverse the tree, I continuously reduce the remaining target sum. When I reach a leaf node, I check whether the remaining target has become zero."

Similar Problems:
- Path Sum II
- Path Sum III
- Maximum Path Sum

Memory Trick:
Target gets smaller as I go down.

==========
BEST TIME TO BUY AND SELL STOCK
==========

Pattern:
Array
Greedy
Dynamic Programming

Recognition Clues:
- Buy once
- Sell once
- Maximize profit
- Find best difference

Think:

Track minimum so far

Key Insight:

Profit today = currentPrice - minimumPriceSeen

Keep track of:

1. Lowest price so far
2. Maximum profit so far

Mistake I Made:
Tried comparing every possible pair of days.

This leads toward $O(n^2)$.

Optimal Solution:
1. Track minimum price seen.
2. For each day:
   - Calculate profit if sold today.
   - Update maximum profit.
   - Update minimum price if necessary.

Complexity:
Time: O(n)
Space: O(1)

Interview Explanation:
"I only need the lowest price before today. For each day I calculate the profit I would make if I sold today and update the best answer."

Similar Problems:
- Best Time to Buy and Sell Stock II
- Maximum Subarray
- Sliding Window Maximum

Memory Trick:
Buy low.
Sell high.
Track minimum.

==========
SINGLE NUMBER
==========

Pattern:
Bit Manipulation

Recognition Clues:
- Every number appears twice except one
- Constant space required
- Unique number must be found

Think:
XOR

Key Insight:

XOR properties:

a ^ a = 0

a ^ 0 = a

Duplicates cancel each other.

Only the unique number remains.

Mistake I Made:
Did not use the XOR trick.

Considered extra storage instead.

Optimal Solution:
Initialize answer = 0

For each number:

answer ^= number

Final answer contains the unique number.

Complexity:
Time: O(n)
Space: O(1)

Interview Explanation:
"Because XOR cancels identical values, every duplicate disappears and only the unique value remains."

Similar Problems:
- Missing Number
- Single Number II
- Bit Manipulation Questions

Memory Trick:
Duplicates disappear with XOR.

==========
LINKED LIST CYCLE
==========

Pattern:
Linked List
Two Pointers
Floyd's Cycle Detection

Recognition Clues:
- Detect cycle
- Loop exists
- Repeated nodes
- Constant space preferred

Think:
Slow and Fast Pointers

Key Insight:

Slow pointer:

Moves 1 step

Fast pointer:

Moves 2 steps

If a cycle exists:

They must eventually meet.

Mistake I Made:
Forgot Floyd's Cycle Detection Theorem.

Optimal Solution:
1. Create slow and fast pointers.
2. Move slow by one.
3. Move fast by two.
4. If they meet:
   - Cycle exists.
5. If fast reaches null:
   - No cycle.

Complexity:
Time: O(n)
Space: O(1)

Interview Explanation:
"If there is a cycle, the faster pointer eventually laps the slower pointer and they meet."

Similar Problems:
- Find Cycle Start
- Happy Number
- Circular Array Loop

Memory Trick:
Fast catches slow.

==========
MAJORITY ELEMENT

==========

Pattern:
Array
Counting
Boyer-Moore Voting

Recognition Clues:
- Element appears more than n/2 times
- Constant space preferred
- Majority guaranteed

Think:
Voting Algorithm

Key Insight:

Majority element survives cancellations.

Whenever two different numbers appear:

Cancel them out.

The majority element remains.

Mistake I Made:
Used a hash map to count frequencies instead of Boyer-Moore Voting.

Optimal Solution:
Keep:

candidate
count

For each number:

```
If count == 0:
    candidate = current

If current == candidate:
    count++

Else:
    count--
```

Final candidate is the majority element.

Complexity:
Time: O(n)
Space: O(1)

Interview Explanation:
"The majority element appears more than half the time, so it survives all pairwise cancellations."

Similar Problems:
- Majority Element II
- Top K Frequent Elements
- Frequency Counting

Memory Trick:
Majority survives cancellation.

==========
REVERSE LINKED LIST
==========

Pattern:
Linked List
Two Pointers
Recursion

Recognition Clues:
- Reverse node connections
- Reverse linked list
- Pointer manipulation

Think:
Save next before changing pointers

Key Insight:

Three pointers:

prev
curr
next

Always save next first.

Mistake I Made:
Overwrote next before saving it.
Lost track of the remaining list.

Optimal Solution:
1. Save next node.
2. Reverse current pointer.
3. Move prev forward.
4. Move curr forward.
5. Repeat.

Complexity:
Time: O(n)
Space: O(1) iterative
Space: O(n) recursive

Interview Explanation:

"I keep track of the previous node, current node, and next node. After saving the next node, I reverse the pointer and continue."

Similar Problems:
- Reverse Linked List II
- Palindrome Linked List
- Reorder List

Memory Trick:
Save.
Reverse.
Move.

==========
CONTAINS DUPLICATE
==========

Pattern:
Array
Hash Table
Sorting

Recognition Clues:
- Detect duplicates
- Fast lookup required
- Membership checking

Think:
Hash Set

Key Insight:

If a number is already in the set:

Duplicate found.

Mistake I Made:
Tried nested loops instead of using a hash set.

Optimal Solution:
1. Create empty hash set.
2. For each number:
   - If already exists:
     return true
   - Otherwise add it
3. Return false

Complexity:
Time: O(n)
Space: O(n)

Alternative:
Sort first.

Time: O(n log n)
Space: O(1)

Interview Explanation:
"As I scan through the array, I keep a set of numbers I've already seen. If I encounter one again, I immediately know a duplicate exists."

Similar Problems:
- Two Sum
- Group Anagrams
- Longest Consecutive Sequence

Memory Trick:
Seen before?
Duplicate.

==========
REMOVE LINKED LIST ELEMENTS
==========

Pattern:
Linked List
Dummy Node

Recognition Clues:
- Delete nodes
- Remove target value
- Head node might be removed

Think:
Dummy Node

Key Insight:

Create:

dummy -> head

This makes deleting the head node easy.

Mistake I Made:
Forgot to handle removal of the head node.
Forgot consecutive target values.

Optimal Solution:
1. Create dummy node.
2. Start at dummy.
3. If next node equals target:
    skip it
4. Otherwise move forward.
5. Return dummy.next

Complexity:
Time: O(n)
Space: O(1) iterative
Space: O(n) recursive

Interview Explanation:
"Using a dummy node allows me to treat deleting the head node exactly the same as deleting any other node."

Similar Problems:
- Remove Duplicates From Sorted List
- Delete Node in Linked List
- Partition List

Memory Trick:
Head can disappear.
Use a dummy node.